Reviewer Pack — Minimal Rank of Modular Forms vs Boolean Tensor Product Valu…

Entry e6c42ce8d456 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 09:47:11 UTC

Minimal Rank of Modular Forms vs Boolean Tensor Product Valuations

Entry ID: `e6c42ce8d456` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 09:47:11 UTC

1. Conjecture

Field A (mathematical branch): Modular Form Theory **Field B** (complexity object): Communication Complexity (Boolean Tensor Product)

Statement:

> ['For every Boolean function f: {0,1}^n -> {0,1}, the minimal rank of its associated modular form M_f in the space of level N = n! and weight k = 2 is upper-bounded by the number of distinct tensor product valuations required to represent f.', 'Equivalently, for a given modular form M_f with minimal rank r(M_f) ≤ N, there exists at most 2^r(M_f) distinct boolean functions that can be represented by M_f using tensor product valuations.']

Rationale (proposer's reasoning):

> ['Modular forms are deeply connected to number theory and have applications in various mathematical fields. Exploring their relationship with communication complexity may reveal new insights into the structure of boolean functions.', 'Boolean tensor products are a fundamental concept in communication complexity, which measures the amount of information needed for two parties to agree on a function. The conjecture suggests that the complexity of representing a modular form using tensor product valuations provides a lower bound on the communication complexity.']

Taxonomy category: `communication_complexity_modular_form` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e1676db98c26bc85`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all generated boolean functions f, the minimal rank r(M_f) of the associated modular form M_f does not exceed twice the number of distinct tensor product valuations required to represent f.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank of modular forms" AND "Boolean tensor product valuation" - "modular form theory" AND "communication complexity" - "Boolean function" AND "tensor product valuation" AND "upper bound"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_boolean_function(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def boolean_tensor_product(f, g):
    n = len(f)
    m = len(g)
    result = []
    for x in range(n):
        for y in range(m):
            if x < n and y < m:
                result.append(f[x] * g[y])
            else:
                result.append(0)  # Handle out-of-range indices by appending 0
    return result

def count_distinct_tensor_product_valuations(boolean_function):
    n = int(math.log2(len(boolean_function)))
    valuations = set()
    for i in range(1 << n):
        f = boolean_function[i:i + n]
        g = boolean_function[n + i:n + 2 * n]
        valuations.add(tuple(boolean_tensor_product(f, g)))
    return len(valuations)

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        boolean_function = generate_boolean_function(n)
        M_f = boolean_function
        r_M_f = len(M_f)  # Minimal rank is the length of the function
        num_valuations = count_distinct_tensor_product_valuations(boolean_function)

        if r_M_f > 2 * num_valuations:
            return {
                "metric_name": "minimal_rank",
                "metric_value": r_M_f,
                "instances_tested": n,
                "conjecture_holds": False,
                "counterexample": f"n={n}, minimal rank {r_M_f} > 2 * {num_valuations}"
            }

        results.append(r_M_f)

    mean = sum(results) / len(results)
    std_dev = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    support_fraction = len([x for x in results if x <= 2 * num_valuations]) / len(results)

    return {
        "metric_name": "minimal_rank",
        "metric_value": mean,
        "instances_tested": sum(n_values),
        "conjecture_holds": support_fraction >= 0.8,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2**i + 1 for i in range(5, 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    all_results = [r for r in results if "conjecture_holds" in r and r["conjecture_holds"]]
    support_fraction = len(all_results) / len(results)

    if all(r["conjecture_holds"] for r in all_results):
        print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in all_results)/len(all_result
    elif any(not r["conjecture_holds"] for r in all_results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"n={n_values[0]}, minimal rank {results[0]['metr
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
IAL: {'metric_name': 'minimal_rank', 'metric_value': 32, 'instances_tested': 5, 'conjecture_holds': 
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 32, 'instances_tested': 5, 'conjecture_holds'
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 32, 'instances_tested': 5, 'conjecture_holds'
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 32, 'instances_tested': 5, 'conjecture_holds'
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 32, 'instances_tested': 5, 'conjecture_holds'
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 32, 'instances_tested': 5, 'conjecture_holds'
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 32, 'instances_tested': 5, 'conjecture_holds'
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 32, 'instances_tested': 5, 'conjecture_holds'
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 32, 'instances_tested': 5, 'conjecture_holds'

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ef22e642.py", line 91, in <module>
    print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in all_results)/len(all_results)} std=
sum(r['metric_value'] for r in all_results)/len(all_results)) ** 2 for r in all_results) / len(all_res
                                          ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^~~~~~~~~~~~~~~~~~~~~~~~
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data that would allow for a definitive conclusion on the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11703
2	preregistration	ollama_remote	glm4:latest	0	0	5446
3	novelty	ollama_remote	glm4:latest	0	0	4596
4	novelty	ollama_remote	glm4:latest	0	0	8962
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13132
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8750
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9720
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10277
9	judge	ollama_remote	glm4:latest	0	0	10295

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 82881 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e6c42ce8d456.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e6c42ce8d456.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e6c42ce8d456.tar.gz` (if generated)